Лабораторная работа №7: Разработка декораторов для кэширования и логирования, адаптеров для внешних банковских API

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 7 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Применить структурные паттерны проектирования GoF (**Adapter**, **Decorator** и **Facade**) для интеграции разнородных внешних сервисов и динамического расширения возможностей платежного шлюза городской транспортной системы Avtobys. В реальной эксплуатации: 1. Валидатор должен взаимодействовать с внешним банковским эквайрингом (например, процессинговый центр Halyk / Kaspi). Внешний банк предоставляет несовместимый API со своими именами методов, форматами ошибок и копейками вместо тенге. Для интеграции мы используем паттерн **Adapter**. 2. Сетевые вызовы к банковскому шлюзу требуют сквозного мониторинга: замера времени выполнения, аудита попыток списания и автоматических повторов при кратковременных сетевых сбоях (Retry). Эти сквозные обязанности реализуются через цепочку **Декораторов**. 3. Весь сложный сценарий обслуживания пассажира на турникете объединяется за простым и надежным **Фасадом**.

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab07_structural_patterns/:

```
lab07_structural_patterns/
├── adapter/
│   ├── __init__.py
│   ├── external_bank.py  # Эмулятор несовместимой сторонней банковской библиотеки
│   └── bank_adapter.py   # Класс BankApiAdapter (соответствует PaymentGateway)
├── decorator/
│   ├── __init__.py
│   ├── base.py           # Протокол PaymentGateway и базовый CoreGateway
│   └── wrappers.py       # Декораторы: TimingDecorator, RetryDecorator, CacheDecorator
├── facade/
│   ├── __init__.py
│   └── terminal_facade.py# TransitTerminalFacade
├── tests/
│   ├── __init__.py
│   ├── test_adapter.py   # Тестирование адаптера
│   ├── test_decorators.py# Тестирование цепочки декораторов
```

```
│   └── test_facade.py    # Сквозной тест фасада
└── README.md             # UML-диаграмма структуры классов
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Реализация BankApiAdapter для стороннего внешнего API	5
Блок 2	Разработка цепочки декораторов (Timing, Retry, Cache)	7
Блок 3	Проектирование TransitTerminalFacade со сквозным сценарием	5
Блок 4	Тестирование юнит-тестами pytest с проверкой моков и исключений	3

---

3. Практические задания

Блок 1. Реализация паттерна Adapter (5 баллов)

В файле adapter/external_bank.py представлен класс стороннего банка:

```
class LegacyBankClient:
    def execute_payment_kzt(self, source_pan: str, dest_account: str, amount_tiyn: int) -> dict:
        """Принимает сумму в тиынах (1 тенге = 100 тиын). Возвращает словарь с 'auth_code'."""
        ...
```

В файле adapter/bank_adapter.py напишите класс BankApiAdapter, реализующий внутренний контракт:

```
class PaymentGateway(Protocol):
    def charge(self, card_id: str, amount: Decimal) -> str:
        """Принимает сумму в Decimal (тенге). Возвращает строковый ID транзакции."""
        ...
```

Адаптер должен корректно переводить Decimal в тиыны, перехватывать банковские сетевые ошибки и оборачивать их в наше доменное исключение PaymentGatewayError.

Блок 2. Разработка цепочки Декораторов (7 баллов)

В decorator/wrappers.py спроектируйте три декоратора:

1. **TimingDecorator**:
    - Замеряет время выполнения метода charge с помощью time.perf_counter().
    - Записывает метрику в словарь latency_metrics: list[float].
2. **RetryOnFailureDecorator**:
    - Если вызов метода падает с NetworkTimeoutError, автоматически повторяет попытку до N раз (по умолчанию 3) с экспоненциальной паузой.
3. **StopListCacheDecorator**:

- Кэширует статусы карт в локальной памяти. Если карта уже отмечена как заблокированная, мгновенно отклоняет операцию без обращения к внешнему шлюзу.

Продемонстрируйте сборку цепочки:

```
gateway = TimingDecorator(
    RetryOnFailureDecorator(
        BankApiAdapter(external_client, transit_account),
        max_retries=3
    )
)
```

Блок 3. Проектирование Facade (5 баллов)

В facade/terminal_facade.py реализуйте фасад:

```
class TransitTerminalFacade:
    def __init__(
        self,
        reader: CardReader,
        gateway: PaymentGateway,
        tariff: TariffEngine,
        hardware: HardwareFeedback
    ):
        ...

    def tap(self) -> TapResponse:
        """
        1. Считывает карту через reader.
        2. Рассчитывает тариф через tariff.
        3. Выполняет списание через gateway.
        4. Подает звуковой/световой сигнал через hardware.
        5. Возвращает объект TapResponse(success=True, fare=fare, tx_id=tx_id).
        """
```

Блок 4. Автоматическое тестирование (pytest) (3 балла)

1. Проверьте адаптер: передача Decimal("150.50") конвертируется во внешнем вызове ровно в 15050 тиын.
2. Проверьте RetryDecorator: если шлюз падает с ошибкой 2 раза и успешен на 3-й, итоговая операция завершается успешно с 3 вызовами.
3. Проверьте TransitTerminalFacade: успешное прикладывание возвращает TapResponse(success=True).

```
pytest tests/ -v
```

---

4. Контрольные вопросы для защиты работы

1. Чем отличается отношение между объектами в паттерне Decorator от отношения в паттерне Proxy?
2. Почему использование Декоратора предпочтительнее добавления логики логирования и повторов прямо в базовый класс шлюза?
3. Как паттерн Facade соотносится с архитектурным принципом разделения ответственностей (SRP)?

4. Каким образом паттерн Adapter позволяет соблюдать принцип независимости доменного слоя от сторонних библиотек?